

Political Alpha Tracker — Code Walkthrough & Implementation Details

This document provides a deep dive into the actual code structure, algorithms, and technical implementation of the Political Alpha Tracker. While the `PROJECT_DOCUMENTATION.md` covers the theory and architecture, this guide is for developers who want to understand *how* the code works under the hood.

1. Directory Structure

```
Political_Alpha_Tracker/
├── main.py                # The main entry point (CLI script)
├── app.py                 # The Streamlit dashboard
├── src/                   # Core application logic
│   ├── config.py         # Global settings & constants
│   ├── pipeline_orchestrator.py # Glues all modules together
│   ├── cache_manager.py  # SQLite Database operations
│   ├── graph_manager.py  # NetworkX graph operations
│   ├── bse_monitor.py    # BSE API scraper & regex parser
│   ├── alpha_engine.py   # Gemini NLP & Conviction Scoring
│   ├── technical_analyzer.py # RSI, MACD, SMA, OBV calculations
│   ├── portfolio_manager.py # Paper trading logic (Kelly, SL, taxes)
│   ├── financial_screener.py # yfinance fundamental checks
│   ├── donor_ingester.py # Electoral bond CSV parsers
│   ├── entity_resolver.py # RapidFuzz string matching
│   ├── backtest.py       # Event study & ML optimization
│   └── ... (various other monitors and scrapers)
└── data/                 # Local storage (SQLite, JSON, CSVs)
```

2. Core Execution Flow (`main.py` -> `PipelineOrchestrator`)

When the cron job runs `python main.py` , the following sequence executes within `src/pipeline_orchestrator.py` :

1. **Initialization:** `PipelineOrchestrator` initializes `CacheManager` , `BSEMonitor` , `AlphaEngine` , `PaperTrader` , etc.
2. `run_daily_pipeline()` : The main function coordinates the workflow.
3. **Universe Build:** It fetches the monitored scrip codes from `CacheManager` .
4. **Virtual Sells:** `self.paper_trader.execute_sells()` runs *first* to check if any existing positions have hit their ATR trailing stop-loss or 90-day time-stop.
5. **BSE Scan:** `self.bse_monitor.scan_watchlist(scrip_codes)` fetches the day's announcements.
6. **Event Processing loop:**
 - Iterates through detected contract events.
 - Calls `financial_screener` to verify the company's fundamentals (D/E, OCF).
 - Calls `graph_manager.alpha_query(cin)` to traverse the graph and find political links.
 - Calls `alpha_engine.calculate_conviction_score(...)` to get the final score (0-13.5).
 - Calls `technical_analyzer.analyze()` to get entry timing (+/- score adjustment).
 - If total score ≥ 4.0 , it calls `notifier.send_alpha_alert()` and `paper_trader.execute_buy()` .
7. **Graph Save:** Finally, `graph_manager.save()` persists any new graph nodes to `data/graph.json` .

3. Database Layer (`src/cache_manager.py`)

The `CacheManager` class is a singleton-like interface to `data/cache.sqlite` .

Key Design Patterns:

- **WAL Mode:** The database runs in Write-Ahead Logging mode (`PRAGMA journal_mode=WAL;`). This is critical because the Streamlit dashboard (`app.py`) reads from the DB concurrently while the background pipeline (`main.py`) writes to it. WAL mode prevents database is locked errors.
- **Context Managers:** Connections are always handled using `with self._connect() as conn:` to ensure they are closed properly.
- **Schema:**
 - `virtual_portfolio` stores open paper trades.
 - `trade_history` stores closed paper trades with calculated PnL.

- `alpha_graph` stores pre-calculated alpha scores for fast dashboard rendering.

4. The Knowledge Graph (`src/graph_manager.py`)

The graph is built using `networkx.DiGraph()` (Directed Graph).

Node Types: `ListedCompany` , `Director` , `DonorCompany` , `ElectoralTrust` , `PoliticalParty` .

The Alpha Query Algorithm (`alpha_query` method):

This is the most computationally intensive part of the code. Given a starting company's CIN:

```
def alpha_query(self, cin: str) -> list[dict]:
    # 1. Find the listed company node
    start_node = f"company:{cin}"
    if start_node not in self.G: return []

    connections = []
    # 2. Find all political parties
    parties = [n for n, d in self.G.nodes(data=True) if d.get('node_type') == 'PoliticalParty']

    for party in parties:
        # 3. Find all simple paths between the company and the party (max length 4 edges)
        paths = list(nx.all_simple_paths(self.G, source=start_node, target=party, cutoff=4))

        for path in paths:
            # 4. Calculate Exclusivity (based on Director node degrees)
            # If a director sits on 50 boards, exclusivity is low. If 2, exclusivity is high.
            # 5. Calculate Proximity (based on path length)
            # 6. Calculate Magnitude (based on DONATED_TO edge weights)

            alpha_score = (0.4 * exclusivity) + (0.3 * proximity) + (0.3 * magnitude)
            connections.append({"path": path, "alpha_score": alpha_score})

    return sorted(connections, key=lambda x: x["alpha_score"], reverse=True)
```

5. NLP and LLM Integration (src/alpha_engine.py)

The AlphaEngine handles unstructured text.

Parsing Contract Details (parse_contract_details):

BSE announcements are often PDFs. The code:

1. Downloads the PDF.
2. Uses `pypdf` to extract the first 3 pages.
3. Sends the text to the Google Gemini API using `google.genai` client.
4. The prompt forces the LLM to return a strict JSON payload containing the `contract_value_cr` (in crores) and the `issuing_authority_state`.

```
# The LLM prompt asks for exactly this format:  
# {"contract_value_cr": 500.5, "issuing_authority_state": "maharashtra"}
```

If the Gemini API fails, it falls back to a regex parser

```
( r"(?i)rs\.\.?s*(\d+(?:\.\d+)?)s*(?:cr|crore)" ).
```

6. Technical Analysis Engine (src/technical_analyzer.py)

Implemented purely using `numpy` and `pandas` (no heavy dependencies like `TA-Lib`).

Indicator Implementations:

- **RSI (Relative Strength Index):** Uses Wilder's smoothing (exponential moving average of gains/losses).
- **MACD (Moving Average Convergence Divergence):** $EMA(12) - EMA(26)$. Signal line is $EMA(MACD, 9)$.
- **ATR (Average True Range):** Calculates True Range (max of high-low, high-prev_close, prev_close-low), then applies a 14-period SMA.

The ATR Stop Loss Formula:

```
# Called when calculating the stop-loss for a new paper trade  
atr_stop = current_close_price - (2 * current_atr_14)
```

7. Paper Trading Logic (`src/portfolio_manager.py`)

The `PaperTrader` handles the simulation logic realistically.

`execute_buy()` :

1. Calculates **Kelly Criterion** sizing based on the Conviction Score (e.g., Score ≥ 8.0 gets 25% allocation, Score ≥ 4.0 gets 5%).
2. Applies a **0.5% Slippage** penalty (execution price = current price * 1.005).
3. Deducts capital and inserts into `virtual_portfolio`.

`execute_sells()` :

1. Fetches all open positions.
2. Checks current price via `yfinance`.
3. If `current_price < atr_stop` OR `days_held >= 90` :
4. Triggers sell.
5. Calculates Taxes: STT (0.1%), Exchange Txn Chg (0.00325%), SEBI Chg, DP Chg (₹15.93 flat).
6. Calculates net PnL and inserts into `trade_history`.
7. Deletes from `virtual_portfolio`.

8. Dashboard Implementation (`app.py`)

Built with **Streamlit**.

Key Technical Details:

- `st.cache_data` : Extensively used to cache heavy SQLite queries (like loading the full trade history) for 5-10 minutes to prevent DB lag.
- **Plotly**: Used in the "Technical Analysis" tab. It creates a 4-row sub-plot (Row 1: Candlesticks/SMA, Row 2: RSI, Row 3: MACD, Row 4: OBV).
- **Graph Rendering**: Uses `pyvis.network.Network` to convert NetworkX data into an interactive HTML canvas. Streamlit renders it using `st.components.v1.html()`.
- **GenAI Chat**: The "Chat with Data" tab uses the Gemini API. It sends the SQLite table schemas in the prompt, asks the LLM to generate a SQL query, executes the query via `pandas.read_sql`, and displays the resulting dataframe.

9. Backtesting Engine (src/backtest.py)

The backtesting framework validates the strategy using historical data.

Event Study (_run_event_study):

For every historical contract announcement:

1. Get the date T_0 .
2. Fetch the stock's return at $T+30$, $T+60$, $T+90$, $T+180$ days.
3. Fetch the benchmark (Nifty Smallcap) return for the exact same date ranges.
4. Excess Return (Alpha) = Stock Return - Benchmark Return.

ML Optimization (run_ml_optimization):

Uses `xgboost.XGBClassifier`.

- **Target (y)**: 1 if the 90-day return was $> 5\%$, else 0.
- **Features (x)**: Alpha Score, Materiality %, Volume Z-Score, Technical Score.
- **Validation**: Uses `sklearn.model_selection.TimeSeriesSplit` (Walk-Forward validation) to prevent look-ahead bias (testing data is strictly chronologically *after* training data).

10. Entity Resolution (src/entity_resolver.py)

The most difficult data engineering challenge: matching dirty company names from Electoral Bonds to clean BSE names.

Uses the **RapidFuzz** library (`fuzz.token_set_ratio`).

```
score = fuzz.token_set_ratio("MEGHA ENGINEERING LTD", "Megha Engineering & Infrastructures Limited")
if score > 75: # DONOR_MATCH_SCORE in config.py
    # Match accepted
```

It cleans names first by aggressively stripping suffixes (LTD, LIMITED, PVT, PRIVATE, INC) and standardizing spacing to prevent false negatives.